

GUIA DE NIVELAMENTO EM C

Vetor de structs, ponteiros, funcoes e gerenciamento de estoque

Resolucao comentada da Atividade 1 - Estrutura de Dados Basica - UEMA - 2026.2

Objetivo deste material

Fazer voce entender o codigo, e nao apenas copiar. Cada conceito novo e explicado com uma analogia, depois vemos como ele aparece na memoria e, por fim, percorremos o codigo linha por linha.

Linguagem: C | Limite: 100 produtos | Dados em memoria

1. O que o exercicio realmente esta pedindo

O enunciado pede um pequeno sistema de loja de eletronicos. O programa precisa cadastrar produtos, listar, buscar pelo codigo, vender, atualizar estoque e acumular o valor vendido. Os produtos ficam em memoria dentro de um vetor de structs, com capacidade maxima de 100 registros.

A parte que parece assustadora e que o exercicio junta varios assuntos de C de uma vez: struct, vetor, ponteiro, funcao, parametro, passagem por referencia, laco, decisao e busca sequencial. Separados, eles sao simples. A dificuldade vem de ve-los encaixados ao mesmo tempo.

Ideia central

Imagine uma loja fisica. Cada Produto e uma ficha com codigo, nome, marca, garantia, preco e quantidade. O vetor e uma estante com 100 espacos para essas fichas. A struct Loja e o painel do gerente: ela sabe quantos itens existem no estoque, quanto dinheiro ja foi vendido e onde comeca a estante de produtos.

O que e obrigatorio e o que e desafio

- Obrigatorio: struct Produto e struct Loja.
- Obrigatorio: vetor de Produto com no maximo 100 posicoes.
- Obrigatorio: funcoes para adicionar, listar, buscar e vender.
- Obrigatorio: uso de ponteiros, decisoes e repeticoes.
- Obrigatorio: atualizar estoque e total de vendas.
- Desafio adicional: gerar relatorio com totais, valor estimado e produtos com maior/menor preco e estoque.

2. Dicionario antes do codigo

Termo	Imagem mental
struct	Um molde de ficha. Em vez de guardar somente um numero, voce cria um tipo que agrupa varias informacoes relacionadas.
vetor	Uma sequencia de varias caixas do mesmo tipo, numeradas a partir de 0. produtos[0], produtos[1], produtos[2]...
ponteiro	Uma variavel que guarda um endereco de memoria. Analogia: um papel com o endereco de um deposito, e nao o deposito em si.
&	Operador de endereco. Pergunta: "em que endereco de memoria esta esta variavel?"
*	Em declaracao, significa ponteiro. Em uma expressao, significa acessar o valor que existe no endereco apontado.
.	Acessa um campo quando voce tem a struct diretamente. Ex.: loja.totalEstoque.
->	Acessa um campo quando voce tem um ponteiro para a struct. Ex.: loja->totalEstoque.
funcao	Um pequeno setor especializado. Voce entrega dados para ele, ele executa uma tarefa e pode ou nao devolver um resultado.
parametro	A informacao que uma funcao recebe para conseguir trabalhar.

Termo	Imagem mental
passagem por valor	A funcao recebe uma copia. Alterar a copia nao altera o original.
passagem por referencia com ponteiro	A funcao recebe o endereco. Assim ela consegue chegar ao original e altera-lo.
indice	Numero da posicao de um elemento no vetor. O primeiro indice e 0.
busca sequencial	Olhar produto por produto ate encontrar o codigo desejado.

3. A memoria: a analogia da estante

Quando escrevemos `Produto produtos[100]`, o computador reserva espaco para 100 fichas `Produto` consecutivas na memoria. No inicio nenhuma delas representa um cadastro valido; por isso existe `totalCadastrados`, que diz quantas posicoes do comeco do vetor ja estao ocupadas.

```
produtos:
[0] Produto | [1] Produto | [2] Produto | ... | [99] Produto
^
|
loja.ptrProduto aponta para o inicio da estante
```

Se `totalCadastrados` vale 3, consideramos validos somente `produtos[0]`, `produtos[1]` e `produtos[2]`. A proxima posicao livre e `produtos[3]`. Por isso a linha `int posicao = *totalCadastrados;` funciona.

A struct Loja NAO contem 100 Produtos

O campo `Produto *ptrProduto` e apenas um endereco. O vetor real `produtos[100]` e criado na `main`. Depois fazemos `loja.ptrProduto = produtos;` e a seta passa a apontar para o primeiro `Produto` do vetor. Isso satisfaz o requisito de ponteiros sem precisar usar `malloc`.

4. Ponto, seta, & e * sem misterio

```
Loja loja;
loja.totalEstoque = 0;          // tenho a struct diretamente -> uso ponto

Loja *p = &loja;
p->totalEstoque = 10;          // tenho um ponteiro para a struct -> uso seta
```

As duas atribuicoes acima chegam ao mesmo campo da mesma Loja. A diferenca e o caminho: `loja` e o objeto; `p` e o endereco do objeto.

```
int x = 7;
int *p = &x;

printf("%d", x);    // 7
printf("%d", *p);   // 7

*p = 20;            // altera x atraves do endereco
```

Analogia: `x` e uma casa que guarda o numero 7. `&x` e o endereco da casa. `p` e um papel no qual escrevemos esse endereco. `*p` significa "va ate o endereco escrito no papel e veja/altere o que ha la".

5. O que uma funcao recebe

```
void listarProdutos(Loja *loja, int totalCadastrados)
void adicionarProduto(Loja *loja, int *totalCadastrados)
```

Na listagem, totalCadastrados e apenas lido; uma copia e suficiente. No cadastro, precisamos aumentar esse contador. Por isso adicionarProduto recebe int *totalCadastrados: o endereco do contador original.

Sobre sua duvida do do...while

Funcao nao vira "parametro do do while". A estrutura e do { comandos } while (condicao);. As chamadas de funcao ficam dentro dos comandos. No nosso codigo, a condicao e apenas opcao != 5. Se for verdadeira, o menu repete; se for falsa, termina.

6.Codigo completo resolvido

A versao abaixo cumpre as funcionalidades obrigatorias e inclui o desafio adicional como opcao 6, mantendo a opcao 5 como Sair. O codigo foi compilado com avisos habilitados e testado com cadastros, listagem, busca, venda e relatorio.

```

1  #include <stdio.h>
2
3  #define MAX_PRODUTOS 100
4
5  typedef struct Produto {
6      int codigo;
7      char nome[100];
8      char marca[100];
9      int garantia;
10     float preco;
11     int quantidade;
12 } Produto;
13
14 typedef struct Loja {
15     int totalEstoque;
16     float totalVendas;
17     Produto *ptrProduto;
18 } Loja;
19
20 int buscarIndicePorCodigo(Loja *loja, int totalCadastrados, int codigo) {
21     for (int i = 0; i < totalCadastrados; i++) {
22         if (loja->ptrProduto[i].codigo == codigo) {
23             return i;
24         }
25     }
26     return -1;
27 }
28
29 void adicionarProduto(Loja *loja, int *totalCadastrados) {
30     if (*totalCadastrados >= MAX_PRODUTOS) {
31         printf("Nao ha mais espaco para cadastrar produtos.\n");
32         return;
33     }
34
35     int codigo;
36     printf("\nCodigo do produto: ");
37     scanf("%d", &codigo);
38
39     if (buscarIndicePorCodigo(loja, *totalCadastrados, codigo) != -1) {
40         printf("Esse codigo ja esta cadastrado.\n");
41         return;
42     }
43
44     int posicao = *totalCadastrados;
45     loja->ptrProduto[posicao].codigo = codigo;
46
47     printf("Nome do produto: ");
48     scanf(" %99[^\n]", loja->ptrProduto[posicao].nome);
49
50     printf("Marca: ");
51     scanf(" %99[^\n]", loja->ptrProduto[posicao].marca);
52
53     printf("Garantia em meses: ");
54     scanf("%d", &loja->ptrProduto[posicao].garantia);
55
56     printf("Preco: R$ ");
57     scanf("%f", &loja->ptrProduto[posicao].preco);
58
59     printf("Quantidade em estoque: ");

```

```

60     scanf("%d", &loja->ptrProduto[posicao].quantidade);
61
62     loja->totalEstoque += loja->ptrProduto[posicao].quantidade;
63     (*totalCadastrados)++;
64
65     printf("Produto cadastrado com sucesso.\n");
66 }
67
68 void listarProdutos(Loja *loja, int totalCadastrados) {
69     if (totalCadastrados == 0) {
70         printf("\nNenhum produto foi cadastrado.\n");
71         return;
72     }
73
74     printf("\n===== PRODUTOS CADASTRADOS =====\n");
75
76     for (int i = 0; i < totalCadastrados; i++) {
77         printf("\nProduto %d\n", i + 1);
78         printf("Codigo: %d\n", loja->ptrProduto[i].codigo);
79         printf("Nome: %s\n", loja->ptrProduto[i].nome);
80         printf("Marca: %s\n", loja->ptrProduto[i].marca);
81         printf("Garantia: %d meses\n", loja->ptrProduto[i].garantia);
82         printf("Preco: R$ %.2f\n", loja->ptrProduto[i].preco);
83         printf("Quantidade em estoque: %d\n", loja->ptrProduto[i].quantidade);
84     }
85 }
86
87 void buscarProduto(Loja *loja, int totalCadastrados) {
88     int codigo;
89     printf("\nDigite o codigo do produto: ");
90     scanf("%d", &codigo);
91
92     int indice = buscarIndicePorCodigo(loja, totalCadastrados, codigo);
93
94     if (indice == -1) {
95         printf("Produto nao encontrado.\n");
96         return;
97     }
98
99     printf("\nProduto encontrado:\n");
100    printf("Codigo: %d\n", loja->ptrProduto[indice].codigo);
101    printf("Nome: %s\n", loja->ptrProduto[indice].nome);
102    printf("Marca: %s\n", loja->ptrProduto[indice].marca);
103    printf("Garantia: %d meses\n", loja->ptrProduto[indice].garantia);
104    printf("Preco: R$ %.2f\n", loja->ptrProduto[indice].preco);
105    printf("Quantidade em estoque: %d\n", loja->ptrProduto[indice].quantidade);
106 }
107
108 void venderProduto(Loja *loja, int totalCadastrados) {
109     int codigo;
110     printf("\nCodigo do produto que sera vendido: ");
111     scanf("%d", &codigo);
112
113     int indice = buscarIndicePorCodigo(loja, totalCadastrados, codigo);
114
115     if (indice == -1) {
116         printf("Produto nao encontrado.\n");
117         return;
118     }
119
120     printf("\nProduto: %s\n", loja->ptrProduto[indice].nome);
121     printf("Preco unitario: R$ %.2f\n", loja->ptrProduto[indice].preco);
122     printf("Estoque atual: %d\n", loja->ptrProduto[indice].quantidade);
123
124     int quantidadeVenda;
125     printf("Quantidade que sera vendida: ");
126     scanf("%d", &quantidadeVenda);
127
128     if (quantidadeVenda <= 0) {
129         printf("A quantidade precisa ser maior que zero.\n");
130         return;
131     }
132
133     if (quantidadeVenda > loja->ptrProduto[indice].quantidade) {
134         printf("Quantidade indisponivel em estoque.\n");
135         return;
136     }
137
138     float valorVenda = loja->ptrProduto[indice].preco * quantidadeVenda;
139
140     loja->ptrProduto[indice].quantidade -= quantidadeVenda;
141     loja->totalVendas += valorVenda;

```

```

142     loja->totalEstoque -= quantidadeVenda;
143
144     printf("Venda realizada com sucesso.\n");
145     printf("Valor da venda: R$ %.2f\n", valorVenda);
146 }
147
148 void gerarRelatorio(Loja *loja, int totalCadastrados) {
149     if (totalCadastrados == 0) {
150         printf("\nNao ha produtos cadastrados para gerar o relatorio.\n");
151         return;
152     }
153
154     float valorEstoque = 0.0f;
155     int maiorPreco = 0;
156     int menorPreco = 0;
157     int maiorQuantidade = 0;
158     int menorQuantidade = 0;
159
160     for (int i = 0; i < totalCadastrados; i++) {
161         valorEstoque += loja->ptrProduto[i].preco * loja->ptrProduto[i].quantidade;
162
163         if (loja->ptrProduto[i].preco > loja->ptrProduto[maiorPreco].preco) {
164             maiorPreco = i;
165         }
166
167         if (loja->ptrProduto[i].preco < loja->ptrProduto[menorPreco].preco) {
168             menorPreco = i;
169         }
170
171         if (loja->ptrProduto[i].quantidade > loja->ptrProduto[maiorQuantidade].quantidade) {
172             maiorQuantidade = i;
173         }
174
175         if (loja->ptrProduto[i].quantidade < loja->ptrProduto[menorQuantidade].quantidade) {
176             menorQuantidade = i;
177         }
178     }
179
180     printf("\n===== RELATORIO =====\n");
181     printf("Produtos cadastrados: %d\n", totalCadastrados);
182     printf("Total de itens em estoque: %d\n", loja->totalEstoque);
183     printf("Valor estimado do estoque: R$ %.2f\n", valorEstoque);
184     printf("Total de vendas realizadas: R$ %.2f\n", loja->totalVendas);
185     printf("Produto com maior preco: %s - R$ %.2f\n",
186           loja->ptrProduto[maiorPreco].nome, loja->ptrProduto[maiorPreco].preco);
187     printf("Produto com menor preco: %s - R$ %.2f\n",
188           loja->ptrProduto[menorPreco].nome, loja->ptrProduto[menorPreco].preco);
189     printf("Produto com maior estoque: %s - %d unidade(s)\n",
190           loja->ptrProduto[maiorQuantidade].nome,
191           loja->ptrProduto[maiorQuantidade].quantidade);
192     printf("Produto com menor estoque: %s - %d unidade(s)\n",
193           loja->ptrProduto[menorQuantidade].nome,
194           loja->ptrProduto[menorQuantidade].quantidade);
195 }
196
197 int main(void) {
198     Produto produtos[MAX_PRODUTOS];
199     Loja loja;
200     int totalCadastrados = 0;
201     int opcao;
202
203     loja.totalEstoque = 0;
204     loja.totalVendas = 0.0f;
205     loja.ptrProduto = produtos;
206
207     do {
208         printf("\n===== \n");
209         printf(" SISTEMA DE GERENCIAMENTO DA LOJA\n");
210         printf("===== \n");
211         printf("1 - Adicionar Produto\n");
212         printf("2 - Listar Produtos\n");
213         printf("3 - Buscar Produto pelo Codigo\n");
214         printf("4 - Vender Produto\n");
215         printf("5 - Sair\n");
216         printf("6 - Gerar Relatorio (desafio adicional)\n");
217         printf("Escolha uma opcao: ");
218         scanf("%d", &opcao);
219
220         switch (opcao) {
221             case 1:
222                 adicionarProduto(&loja, &totalCadastrados);
223                 break;

```

```

224         case 2:
225             listarProdutos(&loja, totalCadastrados);
226             break;
227         case 3:
228             buscarProduto(&loja, totalCadastrados);
229             break;
230         case 4:
231             venderProduto(&loja, totalCadastrados);
232             break;
233         case 5:
234             printf("Encerrando o programa.\n");
235             break;
236         case 6:
237             gerarRelatorio(&loja, totalCadastrados);
238             break;
239         default:
240             printf("Opcao invalida.\n");
241     }
242     } while (opcao != 5);
243
244     return 0;
245 }

```

7. Entendendo cada parte antes da leitura linha por linha

7.1 Struct Produto: a ficha de cadastro

```

typedef struct Produto {
    int codigo;
    char nome[100];
    char marca[100];
    int garantia;
    float preco;
    int quantidade;
} Produto;

```

Uma variavel Produto guarda todas essas informacoes juntas. Se p for um Produto, p.codigo, p.preco e p.quantidade pertencem ao mesmo registro. Isso evita manter seis vetores separados e torcer para os indices continuarem sincronizados.

7.2 Struct Loja: o painel do gerente

```

typedef struct Loja {
    int totalEstoque;
    float totalVendas;
    Produto *ptrProduto;
} Loja;

```

totalEstoque guarda o numero total de unidades disponiveis, e nao o numero de tipos de produto. Se ha 3 notebooks e 10 mouses, totalEstoque vale 13. totalVendas guarda dinheiro acumulado. ptrProduto aponta para o vetor onde as fichas Produto estao armazenadas.

7.3 buscarIndicePorCodigo: o funcionario que procura etiquetas

Imagine um funcionario andando pela estante da esquerda para a direita. Em cada ficha ele olha o codigo. Se a etiqueta bate com o codigo pedido, ele diz o numero da gaveta. Se chegar ao fim e nao encontrar, responde -1.

```

for (int i = 0; i < totalCadastrados; i++)
    se codigo da ficha i == codigo procurado -> return i
se terminou o laco -> return -1

```

7.4 adicionarProduto: recepcao do estoque

1. Confere se a estante ainda tem espaco.
2. Le o codigo e verifica se ele ja existe.
3. Escolhe a proxima posicao livre.
4. Preenche todos os campos da ficha.
5. Soma as unidades ao total geral de estoque.

6. Incrementa o contador de produtos cadastrados.

7.5 listarProdutos: inventario visual

Se nao ha produtos, mostra uma mensagem. Caso contrario, percorre do indice 0 ate totalCadastrados - 1 e imprime os campos de cada ficha.

7.6 buscarProduto: perguntar onde esta a ficha

Essa funcao nao reimplementa a busca. Ela reaproveita buscarIndicePorCodigo. Esse reaproveitamento e uma forma simples de modularizacao: uma funcao pequena resolve uma tarefa e outras funcoes usam o resultado.

7.7 venderProduto: o caixa

O caixa encontra o produto, mostra seus dados, pergunta quantas unidades sairao, valida o estoque, calcula o valor financeiro, diminui a quantidade do produto, soma dinheiro em totalVendas e diminui totalEstoque.

```
valorVenda = preco * quantidadeVenda;
produto.quantidade = produto.quantidade - quantidadeVenda;
totalVendas = totalVendas + valorVenda;
totalEstoque = totalEstoque - quantidadeVenda;
```

7.8 gerarRelatorio: procurar os campeoes

Para maior/menor preco e quantidade, guardamos indices. Comecamos supondo que produtos[0] e o maior e o menor. Depois percorremos o vetor: se aparecer alguem maior, trocamos o indice do campeao. Esse padrao aparece muito em Estruturas de Dados.

7.9 main: o gerente do sistema

A main cria o vetor e a Loja, conecta o ponteiro ao vetor, inicializa os totais e entra no menu. O switch decide qual funcao chamar. O do...while faz o menu voltar ate que a opcao seja 5.

8.Codigo linha por linha

A seguir, cada linha nao vazia do programa recebe uma explicacao. As linhas em branco servem apenas para organizacao visual e nao mudam o comportamento do programa.

8.1 Bibliotecas e structs

Linha de C	O que significa
1: #include <stdio.h>	Inclui a biblioteca stdio.h, que fornece printf e scanf. Pense nela como abrir a caixa de ferramentas de entrada e saida antes de usar essas ferramentas.
3: #define MAX_PRODUTOS 100	Cria uma constante simbolica com valor 100. Em vez de espalhar o numero 100 pelo programa, damos a ele um nome. Analogia: e a placa na porta dizendo capacidade maxima = 100.
5: typedef struct Produto {	Comeca a definicao do molde Produto. typedef cria um apelido para que depois possamos escrever Produto em vez de struct Produto.
6: int codigo;	Declara uma variavel inteira chamada codigo. Ela existe apenas dentro do bloco atual.
7: char nome[100];	Declara um vetor de char, isto e, um espaco para armazenar uma string em C. O numero entre colchetes e a capacidade do vetor.
8: char marca[100];	Declara um vetor de char, isto e, um espaco para armazenar uma string em C. O numero entre colchetes e a capacidade do vetor.
9: int garantia;	Declara uma variavel inteira chamada garantia. Ela existe apenas dentro do bloco atual.

Linha de C	O que significa
10: float preco;	Declara uma variavel float chamada preco, usada para numeros com parte decimal.
11: int quantidade;	Declara uma variavel inteira chamada quantidade. Ela existe apenas dentro do bloco atual.
12: } Produto;	Fecha o molde e cria o nome curto Produto. A partir daqui, Produto vira um tipo, como int ou float, mas criado por nos.
14: typedef struct Loja {	Comeca o molde Loja. Ele guarda informacoes gerais da loja e um ponteiro para os produtos.
15: int totalEstoque;	Declara uma variavel inteira chamada totalEstoque. Ela existe apenas dentro do bloco atual.
16: float totalVendas;	Declara uma variavel float chamada totalVendas, usada para numeros com parte decimal.
17: Produto *ptrProduto;	Produto *ptrProduto declara um ponteiro para Produto. Ele nao guarda um produto inteiro; guarda um endereco de memoria de onde ha Produto(s). Analogia: nao e a estante, e uma seta dizendo onde a estante comeca.
18: } Loja;	Fecha o molde Loja e cria o nome curto Loja.

8.2 Funcao buscarIndicePorCodigo

Linha de C	O que significa
20: int buscarIndicePorCodigo(Loja *loja, int totalCadastrados, int codigo) {	Funcao auxiliar que recebe o endereco da Loja, a quantidade de cadastros validos e o codigo procurado. Ela devolve um indice inteiro.
21: for (int i = 0; i < totalCadastrados; i++) {	Percorre os indices 0, 1, 2... ate o ultimo produto realmente cadastrado. Nao percorremos as 100 posicoes se apenas 3 estiverem ocupadas.
22: if (loja->ptrProduto[i].codigo == codigo) {	Compara o codigo do produto atual com o codigo procurado. O operador == e usado porque loja e um ponteiro; depois [i] escolhe uma posicao do vetor e .codigo acessa o campo daquele Produto.
23: return i;	Se encontrou, devolve i. Exemplo: se o produto esta em produtos[4], a funcao retorna 4.
24: }	Fecha o bloco que foi aberto anteriormente.
25: }	Fecha o bloco que foi aberto anteriormente.
26: return -1;	Se o laço terminou sem encontrar, devolve -1. Usamos -1 como sinal especial de "nao encontrado", porque indices validos comecam em 0.
27: }	Fecha o bloco que foi aberto anteriormente.

8.3 Funcao adicionarProduto

Linha de C	O que significa
29: void adicionarProduto(Loja *loja, int *totalCadastrados) {	Funcao de cadastro. Loja *loja permite alterar a Loja original. int *totalCadastrados permite alterar o contador original de produtos cadastrados.
30: if (*totalCadastrados >= MAX_PRODUTOS) {	O * antes de totalCadastrados significa "pegue o valor que esta no endereco recebido". Verifica se esse valor ja chegou a 100.

Linha de C	O que significa
31: printf("Nao ha mais espaco para cadastrar produtos.\n");	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
32: return;	Como a funcao e void, return; apenas encerra a funcao imediatamente, sem devolver um valor.
33: }	Fecha o bloco que foi aberto anteriormente.
35: int codigo;	Declara uma variavel inteira chamada codigo. Ela existe apenas dentro do bloco atual.
36: printf("\nCodigo do produto: ");	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
37: scanf("%d", &codigo);	scanf grava o numero digitado na variavel codigo. O & significa "endereço de codigo": scanf precisa saber onde escrever na memoria.
39: if (buscarIndicePorCodigo(loja, *totalCadastrados, codigo) != -1) {	Chama a busca antes do cadastro. Se o retorno for diferente de -1, algum produto com esse codigo ja existe.
40: printf("Esse codigo ja esta cadastrado.\n");	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
41: return;	Encerra imediatamente uma funcao void. Como void nao devolve valor, nao ha nada depois de return.
42: }	Fecha o bloco que foi aberto anteriormente.
44: int posicao = *totalCadastrados;	A proxima vaga livre e exatamente o numero de produtos ja cadastrados. Se existem 3, as posicoes usadas sao 0,1,2 e a proxima e 3.
45: loja->ptrProduto[posicao].codigo = codigo;	Grava o codigo na ficha Produto que esta na posicao escolhida.
47: printf("Nome do produto: ");	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
48: scanf(" %99[^\n]", loja->ptrProduto[posicao].nome);	Le uma linha de texto com no maximo 99 caracteres. O espaco inicial consome a quebra de linha pendente; [^\n] significa "leia tudo ate a quebra de linha". Nao usamos & porque um vetor de char ja representa o endereco do primeiro caractere.
50: printf("Marca: ");	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
51: scanf(" %99[^\n]", loja->ptrProduto[posicao].marca);	Faz a mesma leitura para a marca.
53: printf("Garantia em meses: ");	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
54: scanf("%d", &loja->ptrProduto[posicao].garantia);	Le a garantia diretamente para o campo garantia do Produto. Aqui usamos & porque o campo e um int comum.
56: printf("Preco: R\$ ");	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
57: scanf("%f", &loja->ptrProduto[posicao].preco);	Le o preco como float diretamente para o campo preco.
59: printf("Quantidade em estoque: ");	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
60: scanf("%d", &loja->ptrProduto[posicao].quantidade);	Le a quantidade inicial em estoque para o Produto.
62: loja->totalEstoque += loja->ptrProduto[posicao].quantidade;	Soma a quantidade desse novo produto ao contador geral de unidades da loja. += significa "valor atual mais alguma coisa".
63: (*totalCadastrados)++;	Aumenta o numero de produtos cadastrados. Os parenteses fazem primeiro *totalCadastrados e depois ++: incrementamos o valor

Linha de C	O que significa
	apontado, nao o ponteiro.
65: printf("Produto cadastrado com sucesso.\n");	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
66: }	Fecha o bloco que foi aberto anteriormente.

8.4 Funcao listarProdutos

Linha de C	O que significa
68: void listarProdutos(Loja *loja, int totalCadastrados) {	Funcao de listagem. Recebe o endereco da Loja para acessar os produtos e uma copia do numero de cadastros.
69: if (totalCadastrados == 0) {	Se o contador vale zero, nao ha ficha valida no vetor para mostrar.
70: printf("\nNenhum produto foi cadastrado.\n");	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
71: return;	Encerra imediatamente uma funcao void. Como void nao devolve valor, nao ha nada depois de return.
72: }	Fecha o bloco que foi aberto anteriormente.
74: printf("\n===== PRODUTOS CADASTRADOS =====\n");	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
76: for (int i = 0; i < totalCadastrados; i++) {	Repete uma vez para cada produto cadastrado.
77: printf("\nProduto %d\n", i + 1);	Mostra "Produto 1", "Produto 2" etc. Usamos i+1 so para apresentacao; o vetor continua indexado a partir de 0.
78: printf("Codigo: %d\n", loja->ptrProduto[i].codigo);	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
79: printf("Nome: %s\n", loja->ptrProduto[i].nome);	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
80: printf("Marca: %s\n", loja->ptrProduto[i].marca);	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
81: printf("Garantia: %d meses\n", loja->ptrProduto[i].garantia);	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
82: printf("Preco: R\$ %.2f\n", loja->ptrProduto[i].preco);	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
83: printf("Quantidade em estoque: %d\n", loja->ptrProduto[i].quantidade);	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
84: }	Fecha o bloco que foi aberto anteriormente.
85: }	Fecha o bloco que foi aberto anteriormente.

8.5 Funcao buscarProduto

Linha de C	O que significa
87: void buscarProduto(Loja *loja, int totalCadastrados) {	Funcao que procura um produto pedido pelo usuario e mostra seus dados.
88: int codigo;	Declara uma variavel inteira chamada codigo. Ela existe apenas dentro

Linha de C	O que significa
	do bloco atual.
89: printf("\nDigite o codigo do produto: ");	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
90: scanf("%d", &codigo);	Le uma informacao digitada pelo usuario e grava no local de memoria indicado pelo argumento correspondente.
92: int indice = buscarIndicePorCodigo(loja, totalCadastrados, codigo);	Guarda em indice o resultado da funcao de busca.
94: if (indice == -1) {	Se o indice recebido for -1, a busca falhou.
95: printf("Produto nao encontrado.\n");	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
96: return;	Encerra imediatamente uma funcao void. Como void nao devolve valor, nao ha nada depois de return.
97: }	Fecha o bloco que foi aberto anteriormente.
99: printf("\nProduto encontrado:\n");	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
100: printf("Codigo: %d\n", loja->ptrProduto[indice].codigo);	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
101: printf("Nome: %s\n", loja->ptrProduto[indice].nome);	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
102: printf("Marca: %s\n", loja->ptrProduto[indice].marca);	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
103: printf("Garantia: %d meses\n", loja->ptrProduto[indice].garantia);	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
104: printf("Preco: R\$ %.2f\n", loja->ptrProduto[indice].preco);	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
105: printf("Quantidade em estoque: %d\n", loja->ptrProduto[indice].quantidade);	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
106: }	Fecha o bloco que foi aberto anteriormente.

8.6 Funcao venderProduto

Linha de C	O que significa
108: void venderProduto(Loja *loja, int totalCadastrados) {	Funcao de venda. Ela precisa modificar estoque e total de vendas, por isso recebe Loja por ponteiro.
109: int codigo;	Declara uma variavel inteira chamada codigo. Ela existe apenas dentro do bloco atual.
110: printf("\nCodigo do produto que sera vendido: ");	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
111: scanf("%d", &codigo);	Le uma informacao digitada pelo usuario e grava no local de memoria indicado pelo argumento correspondente.
113: int indice = buscarIndicePorCodigo(loja, totalCadastrados, codigo);	Localiza a posicao do produto antes de tentar vender.
115: if (indice == -1) {	Comeca uma decisao: o bloco seguinte so e executado se a expressao entre parenteses for verdadeira.
116: printf("Produto nao encontrado.\n");	Mostra uma mensagem na tela usando printf. Os marcadores como %d,

Linha de C	O que significa
	%s e %.2f sao substituidos pelos valores passados depois da string.
117: return;	Encerra imediatamente uma funcao void. Como void nao devolve valor, nao ha nada depois de return.
118: }	Fecha o bloco que foi aberto anteriormente.
120: printf("\nProduto: %s\n", loja->ptrProduto[indice].nome);	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
121: printf("Preco unitario: R\$ %.2f\n", loja->ptrProduto[indice].preco);	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
122: printf("Estoque atual: %d\n", loja->ptrProduto[indice].quantidade);	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
124: int quantidadeVenda;	Cria uma variavel local para guardar quantas unidades o cliente quer comprar.
125: printf("Quantidade que sera vendida: ");	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
126: scanf("%d", &quantidadeVenda);	Le uma informacao digitada pelo usuario e grava no local de memoria indicado pelo argumento correspondente.
128: if (quantidadeVenda <= 0) {	Validacao extra: impede venda de zero ou quantidade negativa.
129: printf("A quantidade precisa ser maior que zero.\n");	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
130: return;	Encerra imediatamente uma funcao void. Como void nao devolve valor, nao ha nada depois de return.
131: }	Fecha o bloco que foi aberto anteriormente.
133: if (quantidadeVenda > loja->ptrProduto[indice].quantidade) {	Compara o pedido com o estoque daquele produto. Se pedir mais do que existe, a venda e recusada.
134: printf("Quantidade indisponivel em estoque.\n");	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
135: return;	Encerra imediatamente uma funcao void. Como void nao devolve valor, nao ha nada depois de return.
136: }	Fecha o bloco que foi aberto anteriormente.
138: float valorVenda = loja->ptrProduto[indice].preco * quantidadeVenda;	Calcula preco unitario vezes quantidade vendida. Esse e o valor monetario da operacao.
140: loja->ptrProduto[indice].quantidade -= quantidadeVenda;	Diminui a quantidade do produto especifico. -= significa "valor atual menos alguma coisa".
141: loja->totalVendas += valorVenda;	Acumula o valor da venda no totalVendas. Esse campo funciona como um cofrinho que guarda a soma de todas as vendas.
142: loja->totalEstoque -= quantidadeVenda;	Diminui do total geral de unidades exatamente a quantidade que saiu.
144: printf("Venda realizada com sucesso.\n");	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
145: printf("Valor da venda: R\$ %.2f\n", valorVenda);	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
146: }	Fecha o bloco que foi aberto anteriormente.

8.7 Desafio: gerarRelatorio

Linha de C	O que significa
148: void gerarRelatorio(Loja *loja, int totalCadastrados) {	Funcao do desafio adicional. Ela calcula os indicadores pedidos no relatorio.
149: if (totalCadastrados == 0) {	Comeca uma decisao: o bloco seguinte so e executado se a expressao entre parenteses for verdadeira.
150: printf("\nNao ha produtos cadastrados para gerar o relatorio.\n");	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
151: return;	Encerra imediatamente uma funcao void. Como void nao devolve valor, nao ha nada depois de return.
152: }	Fecha o bloco que foi aberto anteriormente.
154: float valorEstoque = 0.0f;	Acumulador do valor financeiro do estoque. Comeca em 0.0f; o f diz que o literal e float.
155: int maiorPreco = 0;	Guarda o indice do produto mais caro encontrado ate agora. Comecamos supondo que o primeiro produto (indice 0) e o campeao.
156: int menorPreco = 0;	Mesmo raciocinio para o mais barato.
157: int maiorQuantidade = 0;	Mesmo raciocinio para a maior quantidade.
158: int menorQuantidade = 0;	Mesmo raciocinio para a menor quantidade.
160: for (int i = 0; i < totalCadastrados; i++) {	Varre todos os produtos para somar o valor do estoque e descobrir os maiores/menores.
161: valorEstoque += loja->ptrProduto[i].preco * loja->ptrProduto[i].quantidade;	Soma preco x quantidade de cada produto. Ex.: 3 teclados de R\$100 contribuem R\$300 ao valor do estoque.
163: if (loja->ptrProduto[i].preco > loja->ptrProduto[maiorPreco].preco) {	Compara o preco atual com o preco do campeao guardado em maiorPreco.
164: maiorPreco = i;	Se o atual ganhou, o indice dele vira o novo maiorPreco.
165: }	Fecha o bloco que foi aberto anteriormente.
167: if (loja->ptrProduto[i].preco < loja->ptrProduto[menorPreco].preco) {	Compara para descobrir o menor preco.
168: menorPreco = i;	Atualiza o indice do produto que esta vencendo essa comparacao. Guardamos o indice, nao uma copia do produto inteiro.
169: }	Fecha o bloco que foi aberto anteriormente.
171: if (loja->ptrProduto[i].quantidade > loja->ptrProduto[maiorQuantidade].quantidade) {	Compara para descobrir quem tem mais unidades em estoque.
172: maiorQuantidade = i;	Atualiza o indice do produto que esta vencendo essa comparacao. Guardamos o indice, nao uma copia do produto inteiro.
173: }	Fecha o bloco que foi aberto anteriormente.
175: if (loja->ptrProduto[i].quantidade < loja->ptrProduto[menorQuantidade].quantidade) {	Compara para descobrir quem tem menos unidades em estoque.
176: menorQuantidade = i;	Atualiza o indice do produto que esta vencendo essa comparacao. Guardamos o indice, nao uma copia do produto inteiro.
177: }	Fecha o bloco que foi aberto anteriormente.
178: }	Fecha o bloco que foi aberto anteriormente.
180: printf("\n===== RELATORIO =====\n");	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.

Linha de C	O que significa
181: printf("Produtos cadastrados: %d\n", totalCadastrados);	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
182: printf("Total de itens em estoque: %d\n", loja->totalEstoque);	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
183: printf("Valor estimado do estoque: R\$ %.2f\n", valorEstoque);	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
184: printf("Total de vendas realizadas: R\$ %.2f\n", loja->totalVendas);	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
185: printf("Produto com maior preco: %s - R\$ %.2f\n",	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
186: loja->ptrProduto[maiorPreco].nome, loja->ptrProduto[maiorPreco].preco);	Acessa um produto do vetor por meio do ponteiro armazenado na Loja e altera ou consulta um de seus campos.
187: printf("Produto com menor preco: %s - R\$ %.2f\n",	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
188: loja->ptrProduto[menorPreco].nome, loja->ptrProduto[menorPreco].preco);	Acessa um produto do vetor por meio do ponteiro armazenado na Loja e altera ou consulta um de seus campos.
189: printf("Produto com maior estoque: %s - %d unidade(s)\n",	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
190: loja->ptrProduto[maiorQuantidade].nome,	Acessa um produto do vetor por meio do ponteiro armazenado na Loja e altera ou consulta um de seus campos.
191: loja->ptrProduto[maiorQuantidade].quantidade);	Acessa um produto do vetor por meio do ponteiro armazenado na Loja e altera ou consulta um de seus campos.
192: printf("Produto com menor estoque: %s - %d unidade(s)\n",	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
193: loja->ptrProduto[menorQuantidade].nome,	Acessa um produto do vetor por meio do ponteiro armazenado na Loja e altera ou consulta um de seus campos.
194: loja->ptrProduto[menorQuantidade].quantidade);	Acessa um produto do vetor por meio do ponteiro armazenado na Loja e altera ou consulta um de seus campos.
195: }	Fecha o bloco que foi aberto anteriormente.

8.8 Funcao main e menu

Linha de C	O que significa
197: int main(void) {	Ponto de entrada do programa. A execucao comeca em main.
198: Produto produtos[MAX_PRODUTOS];	Cria de verdade o vetor com capacidade para 100 structs Produto. Pense em uma estante com 100 gavetas de fichas de produto.
199: Loja loja;	Cria uma variavel Loja. Ela sera o painel de controle geral.
200: int totalCadastrados = 0;	Contador de quantas posicoes iniciais do vetor estao ocupadas por produtos validos. Comeca em zero.
201: int opcao;	Guarda a opcao escolhida no menu.
203: loja.totalEstoque = 0;	Inicializa o total de unidades em estoque com zero.
204: loja.totalVendas = 0.0f;	Inicializa o dinheiro acumulado de vendas com zero.
205: loja.ptrProduto = produtos;	Faz ptrProduto apontar para o inicio do vetor produtos. Aqui esta a ligacao entre a Loja e a estante de produtos.

Linha de C	O que significa
207: do {	Comeca um do...while. O bloco do do e executado primeiro; somente depois a condicao do while e testada. Isso garante que o menu apareca pelo menos uma vez.
208: printf("\n=====	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
209: printf(" SISTEMA DE GERENCIAMENTO DA LOJA\n");	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
210: printf("=====	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
211: printf("1 - Adicionar Produto\n");	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
212: printf("2 - Listar Produtos\n");	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
213: printf("3 - Buscar Produto pelo Codigo\n");	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
214: printf("4 - Vender Produto\n");	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
215: printf("5 - Sair\n");	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
216: printf("6 - Gerar Relatorio (desafio adicional)\n");	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
217: printf("Escolha uma opcao: ");	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
218: scanf("%d", &opcao);	Le a opcao e grava em opcao; &opcao fornece o endereco dessa variavel.
220: switch (opcao) {	switch escolhe um caminho conforme o valor de opcao. E como um atendente olhando o numero da senha e encaminhando para o setor correto.
221: case 1:	Case 1: este bloco e escolhido quando opcao vale 1.
222: adicionarProduto(&loja, &totalCadastrados);	Chama adicionarProduto passando &loja e &totalCadastrados. O & entrega os enderecos, permitindo que a funcao altere os originais.
223: break;	break encerra este case para o programa nao cair nos cases seguintes.
224: case 2:	Case 2: este bloco e escolhido quando opcao vale 2.
225: listarProdutos(&loja, totalCadastrados);	Lista os produtos. totalCadastrados vai sem &, portanto a funcao recebe uma copia do numero.
226: break;	Interrompe o case atual do switch e continua a execucao depois do switch.
227: case 3:	Case 3: este bloco e escolhido quando opcao vale 3.
228: buscarProduto(&loja, totalCadastrados);	Busca e exhibe um produto pelo codigo.
229: break;	Interrompe o case atual do switch e continua a execucao depois do switch.
230: case 4:	Case 4: este bloco e escolhido quando opcao vale 4.
231: venderProduto(&loja, totalCadastrados);	Executa a venda e altera a Loja por meio do ponteiro recebido.
232: break;	Interrompe o case atual do switch e continua a execucao depois do switch.

Linha de C	O que significa
233: case 5:	Case 5: este bloco e escolhido quando opcao vale 5.
234: printf("Encerrando o programa.\n");	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
235: break;	Interrompe o case atual do switch e continua a execucao depois do switch.
236: case 6:	Case 6: este bloco e escolhido quando opcao vale 6.
237: gerarRelatorio(&loja, totalCadastrados);	Executa o desafio adicional. Mantivemos a opcao 5 como Sair, como pede o enunciado, e colocamos o relatorio na opcao 6.
238: break;	Interrompe o case atual do switch e continua a execucao depois do switch.
239: default:	default e executado quando a opcao nao corresponde a nenhum case.
240: printf("Opcao invalida.\n");	Mostra uma mensagem na tela usando printf. Os marcadores como %d, %s e %.2f sao substituidos pelos valores passados depois da string.
241: }	Fecha o bloco que foi aberto anteriormente.
242: } while (opcao != 5);	Aqui esta a condicao do do...while: repetir enquanto opcao for diferente de 5. IMPORTANTE: nenhuma funcao e "parametro do do while"; as funcoes estao dentro do bloco, e o while recebe apenas uma expressao logica.
244: return 0;	return 0 informa ao sistema operacional que o programa terminou normalmente.
245: }	Fecha o bloco da funcao main.

9. As expressoes que mais confundem

9.1 loja->ptrProduto[i].codigo

Leia da esquerda para a direita, em etapas:

1. loja: e um ponteiro para uma struct Loja.
2. loja->ptrProduto: entre na Loja apontada e pegue o ponteiro que aponta para o vetor de Produto.
3. loja->ptrProduto[i]: escolha o Produto de indice i.
4. loja->ptrProduto[i].codigo: dentro desse Produto, acesse o campo codigo.

Analogia

loja e o endereco do escritorio do gerente. ->ptrProduto pega o mapa da estante. [i] escolhe a gaveta. .codigo abre a gaveta e le a etiqueta codigo.

9.2 &loja e &totalCadastrados

Na chamada adicionarProduto(&loja, &totalCadastrados), nao enviamos copias. Enviamos os enderecos. A funcao consegue chegar aos objetos originais e modifica-los. Isso e passagem por referencia feita com ponteiros em C.

9.3 *totalCadastrados

Dentro de adicionarProduto, totalCadastrados e um ponteiro. Se voce quer o numero guardado no endereco, usa *. Por isso *totalCadastrados pode valer 0, 1, 2...

```
int total = 3;
int *p = &total;

printf("%d", total); // 3
printf("%d", *p);    // 3

(*p)++;              // total passa a valer 4
```

9.4 Por que scanf usa & em numeros, mas nao em nome?

```
int codigo;
scanf("%d", &codigo);

char nome[100];
scanf(" %99[^\n]", nome);
```

scanf precisa de um endereco para escrever. codigo e uma unica variavel int, entao usamos &codigo para obter seu endereco. nome e um vetor; em muitas expressoes, o nome do vetor ja se transforma no endereco de seu primeiro elemento. Por isso usamos nome, sem &.

9.5 O que significa %99[^\n]?

- O espaco antes do % manda scanf ignorar espacos e quebras de linha pendentes antes de comecar.
- %99 limita a leitura a no maximo 99 caracteres, preservando espaco para o caractere final \0 da string.
- [^\n] significa: aceite qualquer caractere que NAO seja quebra de linha. Assim o nome pode ter espacos.

9.6 =, ==, +=, -= e !=

Operador	Leitura	Exemplo
=	atribuicao	x = 5 coloca 5 dentro de x.
==	comparacao de igualdade	x == 5 pergunta se x vale 5.

<code>!=</code>	diferente	<code>opcao != 5</code> pergunta se <code>opcao</code> e diferente de 5.
<code>+=</code>	somar e guardar	<code>total += 3</code> equivale a <code>total = total + 3</code> .
<code>-=</code>	subtrair e guardar	<code>estoque -= 2</code> equivale a <code>estoque = estoque - 2</code> .
<code>++</code>	incrementar	<code>i++</code> aumenta <code>i</code> em uma unidade.

10. Simulacao manual: o que muda na memoria

Vamos cadastrar dois produtos e depois vender duas unidades do primeiro. Os enderecos abaixo sao apenas ilustrativos; o computador pode usar outros enderecos reais.

Passo A - logo apos iniciar

```
totalCadastrados = 0
loja.totalEstoque = 0
loja.totalVendas = 0.00
loja.ptrProduto -> inicio de produtos[100]
```

Passo B - cadastrar Notebook, 5 unidades, R\$ 3500,50

```
produtos[0] = { codigo:101, nome:"Notebook Gamer", marca:"Lenovo", garantia:12, preco:3500.50,
quantidade:5 }
totalCadastrados = 1
loja.totalEstoque = 5
loja.totalVendas = 0.00
```

Passo C - cadastrar Mouse, 10 unidades, R\$ 150,00

```
produtos[1] = { codigo:202, nome:"Mouse sem fio", marca:"Logitech", garantia:24, preco:150.00,
quantidade:10 }
totalCadastrados = 2
loja.totalEstoque = 15
```

Passo D - vender 2 Notebooks

A busca retorna indice 0. O estoque do Notebook e 5, portanto vender 2 e permitido. `valorVenda = 3500,50 x 2 = 7001,00`.

```
produtos[0].quantidade: 5 -> 3
loja.totalEstoque:      15 -> 13
loja.totalVendas:      0.00 -> 7001.00
```

Repare na diferenca

`totalCadastrados` continua 2. Vender unidades nao apaga o cadastro do produto. So diminui a quantidade. Mesmo que chegue a zero, a ficha do Produto ainda existe no vetor.

11. Por que o relatorio guarda indices?

Suponha que `maiorPreco` vale 2. Isso quer dizer: "o produto que esta vencendo a disputa de maior preco esta em `produtos[2]`". Guardar apenas o indice e barato e simples; quando precisamos do nome ou preco, acessamos `produtos[maiorPreco]`.

```
if (produto_atual.preco > produto_campeao.preco)
    maiorPreco = i;
```

Esse padrao e uma tecnica de varredura: percorremos uma vez o vetor e carregamos conosco a melhor resposta encontrada ate aquele momento.

12. Erros comuns que voce pode encontrar

Esquecer & no scanf de int/float: scanf precisa do endereco para gravar. Sem &, voce pode receber aviso/erro ou comportamento indefinido.

Usar . quando a variavel e ponteiro: Se loja e Loja*, use loja->campo. Se loja e Loja, use loja.campo.

Usar -> quando a variavel nao e ponteiro: O inverso tambem e erro: a seta pressupoe um endereco para uma struct.

Confundir totalCadastrados com totalEstoque: Um conta quantos tipos/registros existem; o outro soma quantas unidades fisicas restam.

Percorrer ate 100 sempre: Funciona em alguns cenarios, mas voce leria posicoes ainda nao cadastradas. O limite correto de dados validos e totalCadastrados.

Esquecer de verificar codigo duplicado: Dois registros com o mesmo codigo tornam a busca ambigua; o exercicio pede impedir isso.

Achar que -1 e um indice: -1 e apenas um sinal de falha. Nunca devemos usar ptrProduto[-1].

Achar que do...while recebe funcao: do...while recebe uma condicao logica no final. As funcoes sao apenas comandos executados dentro do bloco.

Achar que Produto *ptrProduto e um Produto completo: E apenas um endereco. O Produto real esta no vetor produtos.

13. Como pensar para escrever isso sozinho

1. Primeiro desenhe as structs. Pergunte: quais dados pertencem a um Produto? Quais dados sao gerais da Loja?
2. Depois crie o vetor e o contador de posicoes ocupadas.
3. Conecte o ponteiro da Loja ao vetor.
4. Implemente primeiro a busca por codigo, porque cadastro e venda se beneficiam dela.
5. Implemente adicionar e teste com 1 ou 2 produtos.
6. Implemente listar e confira se os dados foram realmente gravados.
7. Implemente buscar usando a funcao auxiliar.
8. Implemente vender e confira as tres mudancas: quantidade do produto, totalEstoque e totalVendas.
9. Somente depois faca o menu. O menu apenas escolhe qual funcao ja pronta chamar.
10. Por ultimo, faca o desafio do relatorio.

14. Mini folha de cola

Sintaxe	Leia assim
Produto p;	cria uma ficha Produto
Produto v[100];	cria um vetor de 100 fichas
p.codigo	acessa campo de uma struct direta
Produto *ptr;	cria ponteiro para Produto
ptr = v;	faz ptr apontar para o inicio do vetor
ptr[i].codigo	acessa codigo do Produto i usando o ponteiro

<code>Loja *loja</code>	parametro que recebe endereco de Loja
<code>loja->totalEstoque</code>	acessa campo da Loja apontada
<code>&x</code>	endereco de x
<code>*p</code>	valor que esta no endereco guardado em p
<code>(*p)++</code>	incrementa o valor apontado
<code>return -1;</code>	sinal convencional de busca sem resultado
<code>for (...)</code>	repete com contador
<code>if (...)</code>	executa se condicao for verdadeira
<code>switch/case</code>	escolhe uma entre varias opcoes
<code>do {...} while (...);</code>	executa pelo menos uma vez e depois decide se repete

15. Exercicios curtos para fixar

Nao olhe o codigo por alguns minutos e tente responder no papel. A ideia e verificar se voce entendeu a estrutura, e nao decorar.

1. Se `totalCadastrados` vale 4, qual e o indice da proxima posicao livre?
2. Qual a diferenca entre `totalCadastrados` e `totalEstoque`?
3. Por que `adicionarProduto` recebe `int *totalCadastrados`, mas `listarProdutos` recebe `int totalCadastrados`?
4. O que significa `loja->ptrProduto[2].preco`?
5. Por que `buscarIndicePorCodigo` devolve -1 quando falha?
6. O que acontece se vender 3 unidades de um produto que tem quantidade 2?
7. Depois de vender todas as unidades de um produto, `totalCadastrados` diminui? Por que?
8. Explique com suas palavras o que `loja.ptrProduto = produtos` faz.
9. Em `do { ... } while (opcao != 5);`, qual parte e a condicao de repeticao?
10. Qual e a diferenca entre `&loja` e `loja` dentro de uma chamada de funcao?

Respostas esperadas

1. 4, porque as posicoes ocupadas sao 0, 1, 2 e 3.
2. `totalCadastrados` conta registros/tipos de produto; `totalEstoque` soma unidades fisicas disponiveis.
3. `Adicionar` precisa alterar o contador original; `listar` apenas precisa ler o valor.
4. Acessar o preco do Produto que esta no indice 2 do vetor apontado por `ptrProduto` dentro da Loja apontada por `loja`.
5. -1 nao e um indice valido e funciona como sinal de "nao encontrei".
6. A venda e recusada e nenhuma quantidade/valor deve ser atualizado.
7. Nao. O cadastro continua existindo; somente a quantidade do produto chega a zero.

- 8. Faz o ponteiro da Loja apontar para o primeiro elemento do vetor produtos.
- 9. opcao != 5.
- 10. &loja e o endereco da variavel Loja; loja e a propria struct quando estamos na main.

16. Observacoes finais sobre esta solucao

Esta implementacao usa somente memoria local e um vetor fixo, exatamente adequado ao limite de 100 produtos. Nao ha arquivo nem banco de dados; ao encerrar o programa, os cadastros somem, pois o exercicio pede armazenamento em memoria.

A entrada numerica pressupoe que o usuario digite numeros validos. Validacao completa de entrada seria um assunto adicional e deixaria o exemplo maior. O foco aqui e compreender structs, vetores, ponteiros, funcoes, busca e atualizacao de registros.

Se voce conseguir explicar em voz alta a relacao entre produtos[100], loja.ptrProduto e totalCadastrados, ja dominou o coracao do exercicio. O restante sao operacoes em cima dessa mesma estrutura.